Data Visualization with ggplot2: Skeleton to Publication-Ready

Instructor: Yuanyuan Wen · Virginia Tech

DATA SCIENCE FOR THE PUBLIC GOOD (DSPG) · JUNE 16, 2026

Our Goals

By the end of this session you will be able to:

- **Produce** publication-ready static figures using ggplot2
- **Choose** the right chart type for your data and the story you want to tell
- **Apply** a refinement workflow: start from a skeleton and build to a polished figure step by step

> You already know how to make plots that work. Today we focus on making them **communicate**.

ggplot2 Syntax — A Quick Review

The Grammar of Graphics

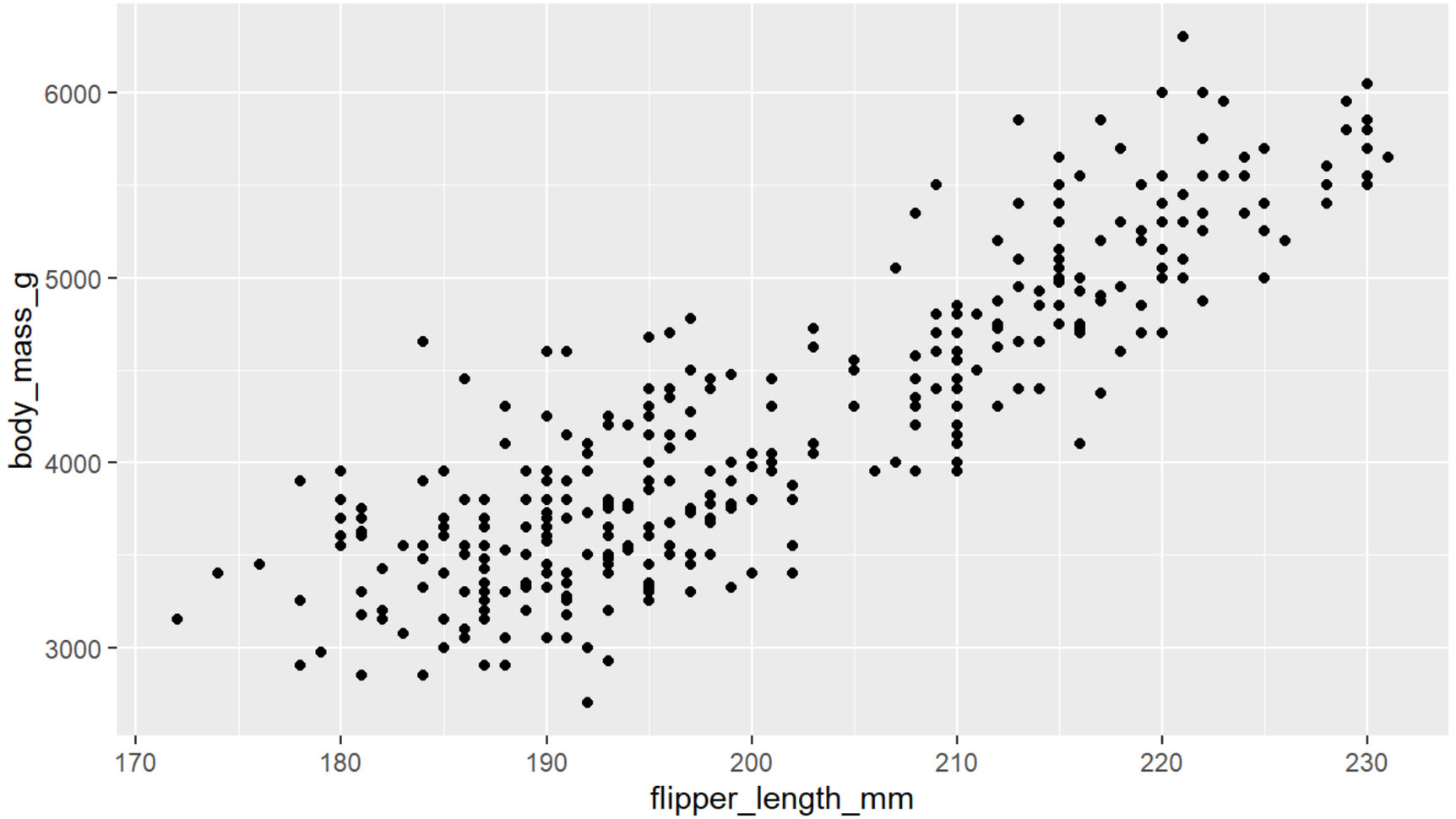
Every ggplot2 call follows the same skeleton:

```
ggplot(data = <DATA>) +
  aes(x = <VAR>, y = <VAR>, color = <VAR>) +
  geom_<TYPE>()
```

Layer	What it controls
`ggplot(data)`	Which dataset
`aes()`	Columns → x, y, color, size, shape
`geom_*()`	The visual mark — point, bar, line, histogram, …

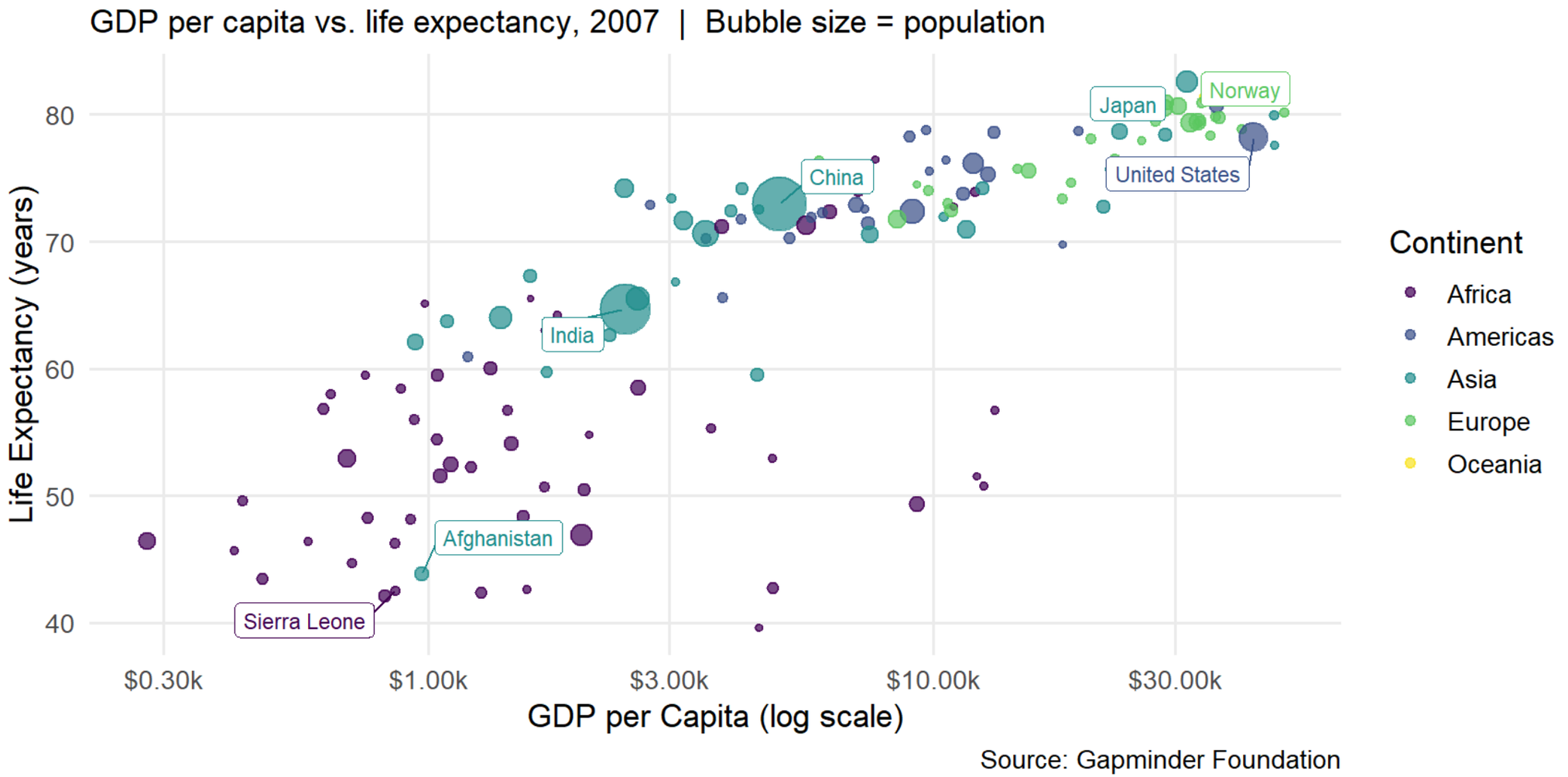
A Minimal ggplot2 Plot

```
ggplot(penguins, aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point()
```



From Skeleton to Publish-Ready

Describe This Plot in One Sentence



What is the one-sentence takeaway from this chart?

The Answer

> **“Wealthier countries tend to live longer, but the relationship flattens at high income levels — and a few wealthy countries still have surprisingly short life expectancies.”**

A good figure has **one clear sentence**. If you cannot write it, the chart is not done yet.

We will build a plot like this together — step by step — using ACS state-level data.

Our Dataset: ACS State-Level Data

Pull from API — run once before class, save locally:

```
library(tidycensus)

state_data <- get_acs(
  geography = "state",
  variables = c(
    median_income = "B19013_001",   # median household income
    pop_25plus    = "B15003_001",   # population 25 and over
    ba            = "B15003_022",   # bachelor's degree
    ma            = "B15003_023",   # master's degree
    prof          = "B15003_024",   # professional degree
    phd           = "B15003_025"    # doctorate
  ),
  year   = 2022,
  output = "wide"
) |>
  mutate(pct_college = (baE + maE + profE + phdE) / pop_25plusE * 100) |>
  left_join(
    tibble(
      NAME   = state.name,
      region = recode(as.character(state.region), "North Central" = "Midwest")
    ),
    by = "NAME"
```

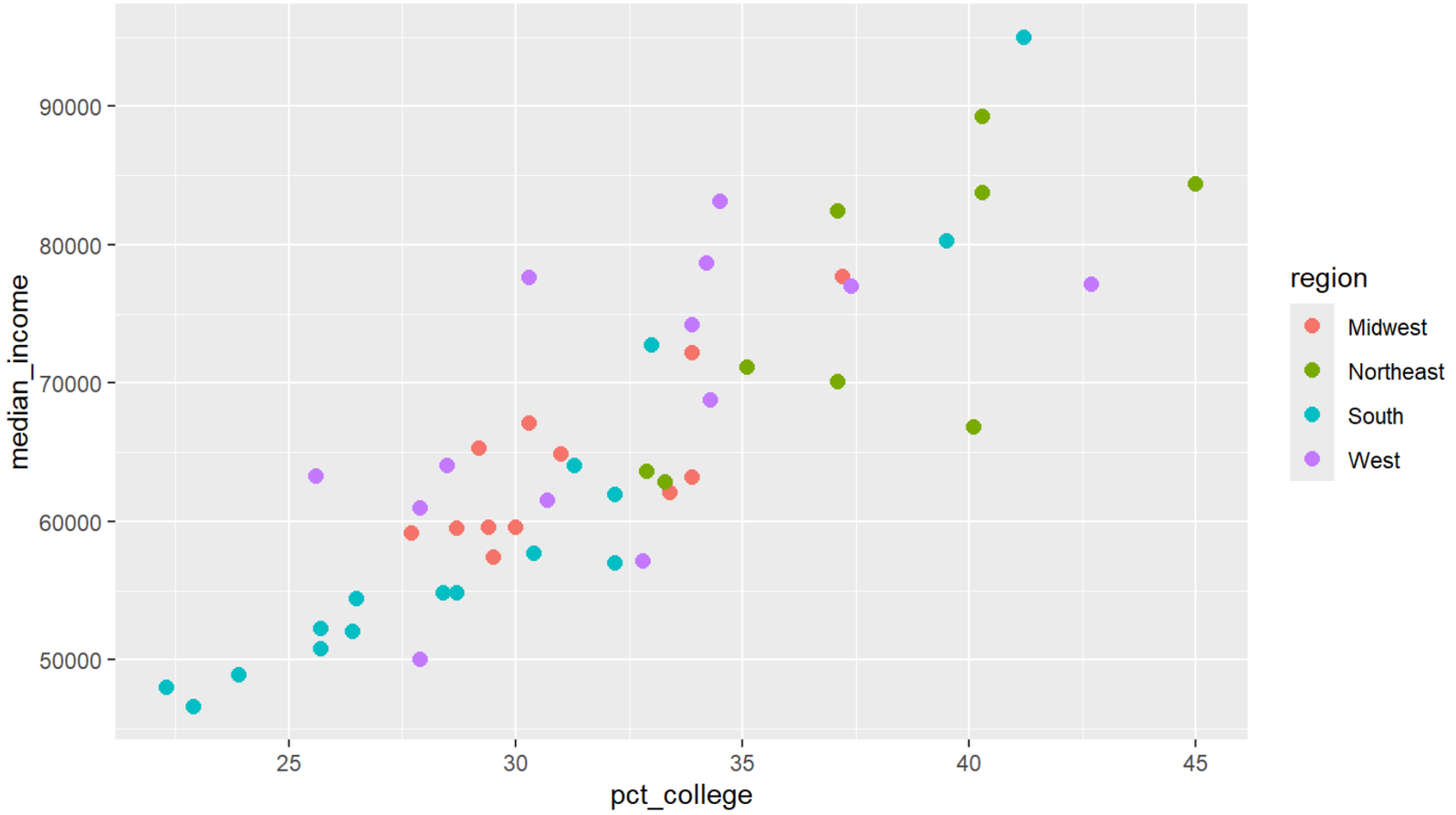
Fallback — if the API is down:

```
state_data <- read_csv("data/state_acs_2022.csv")
```

Step 1 — Skeleton

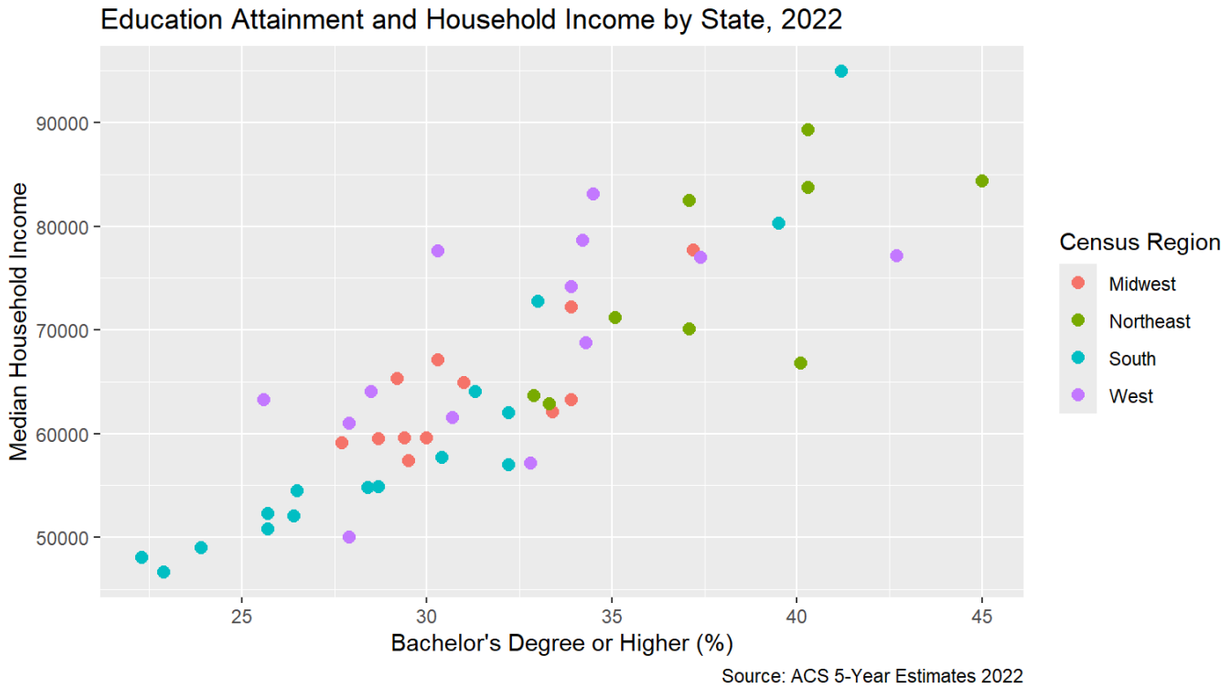
Wire up the data. Confirm the relationship is visible before adding anything else.

```
ggplot(state_data, aes(x = pct_college, y = median_income, color = region)) +
  geom_point(size = 2.5)
```



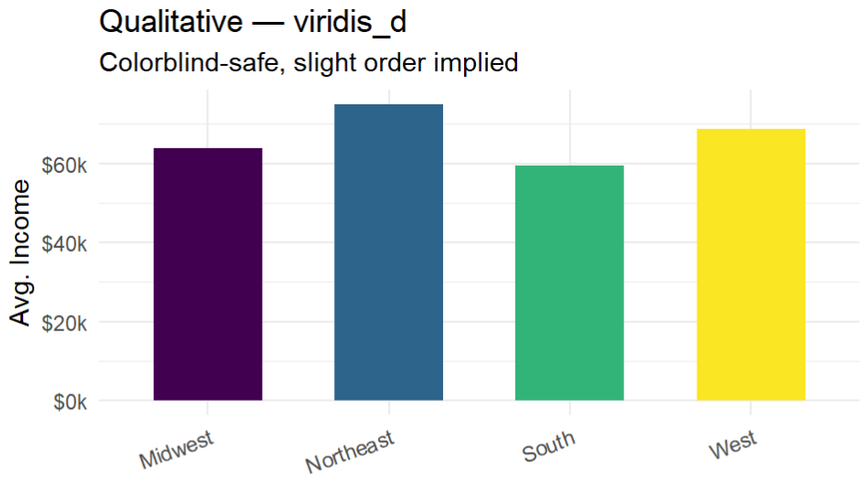
Step 2 — Labels

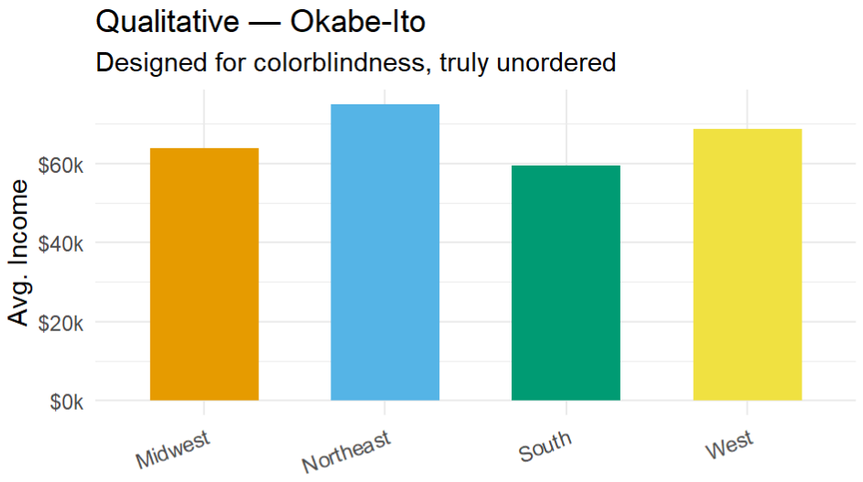
```
ggplot(state_data, aes(x = pct_college, y = median_income, color = region)) +
  geom_point(size = 2.5) +
  labs(
    title   = "Education Attainment and Household Income by State, 2022",
    x       = "Bachelor's Degree or Higher (%)",
    y       = "Median Household Income",
    color   = "Census Region",
    caption = "Source: ACS 5-Year Estimates 2022"
  )
```

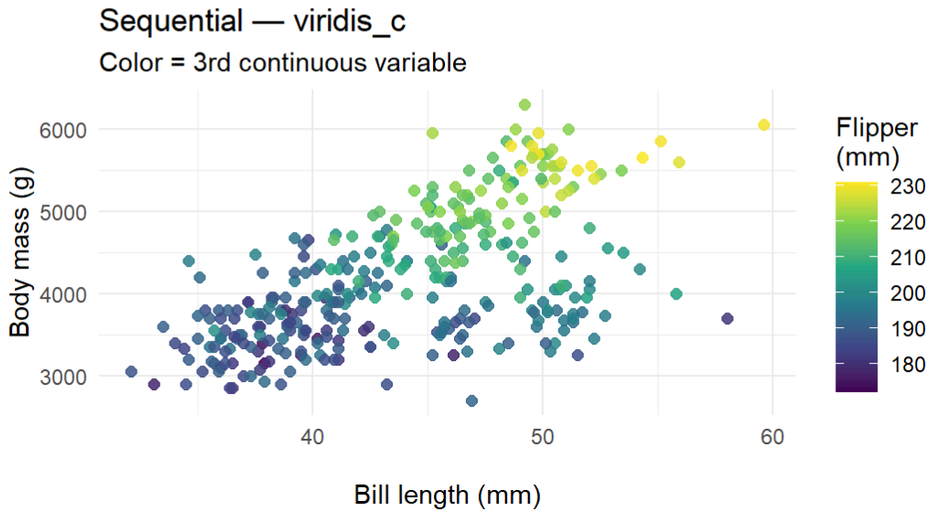


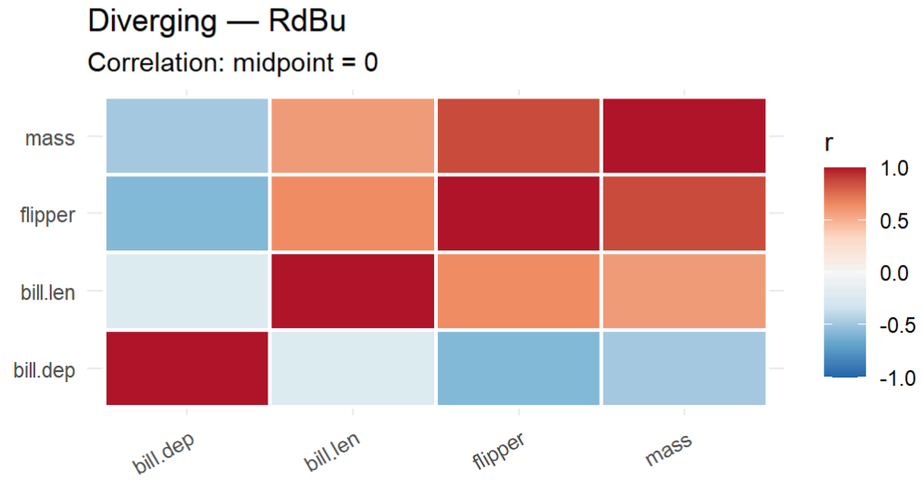
Choosing a Color Palette

Match your palette to your data type:

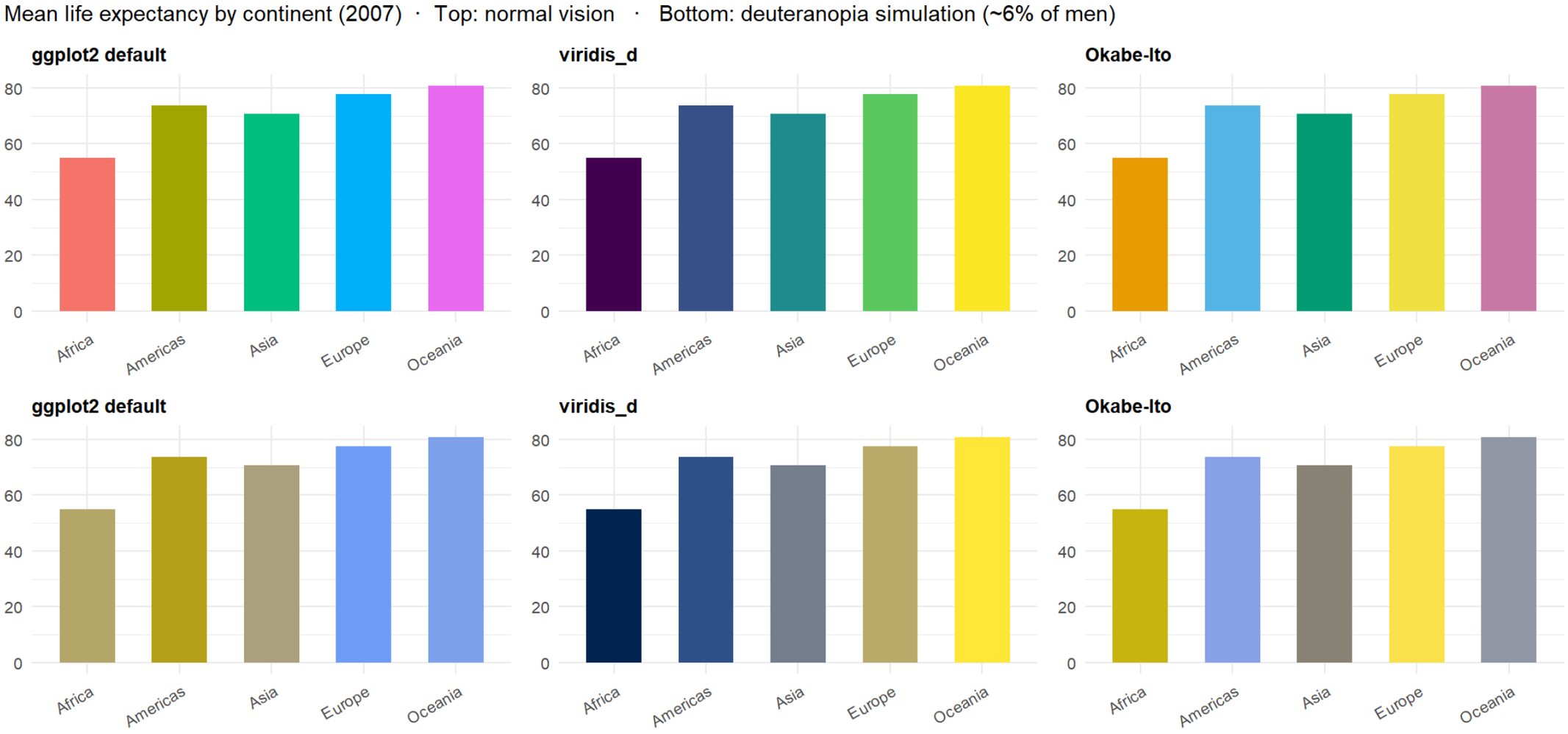








Why Not ggplot2 Default? Colorblind Simulation



Default palette loses distinction under colorblindness. viridis and Okabe-Ito hold up.

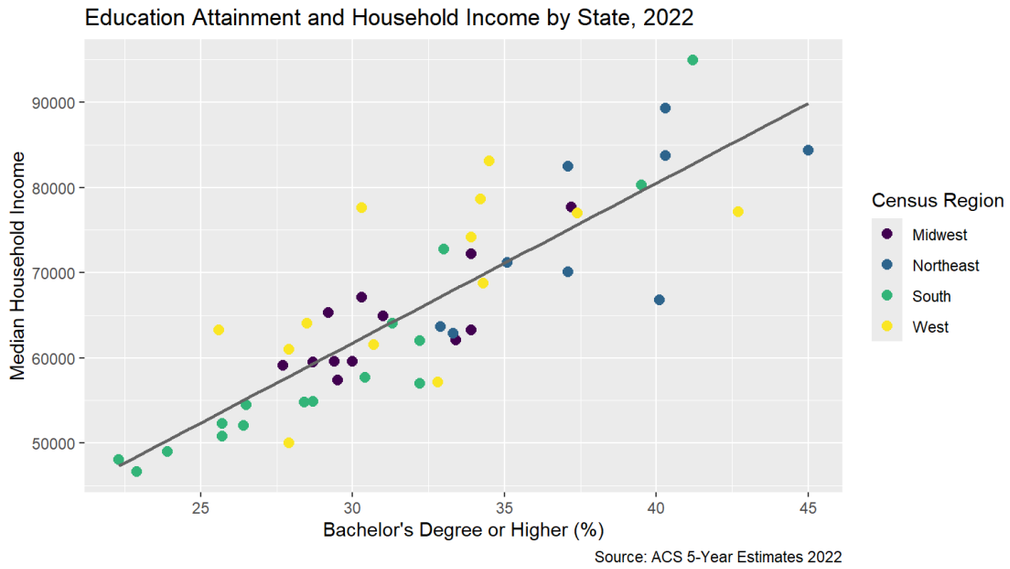
Color Rules for Categorical Palettes

- Max **5–6 colors** — beyond that, readers cannot distinguish
- Prefer **Okabe-Ito** (no order implied) or `viridis_d` (slight order) — both colorblind-safe
- Avoid ggplot2 **default** and **red/green** combinations
- Test in **grayscale** — if it becomes unreadable, it won't print well

Categorical → `scale_fill_manual(values = c("#E69F00", "#56B4E9", "#009E73", ...))` (Okabe-Ito)
Sequential → `scale_color_viridis_c()`
Diverging → `scale_color_distiller(palette = "RdBu")`

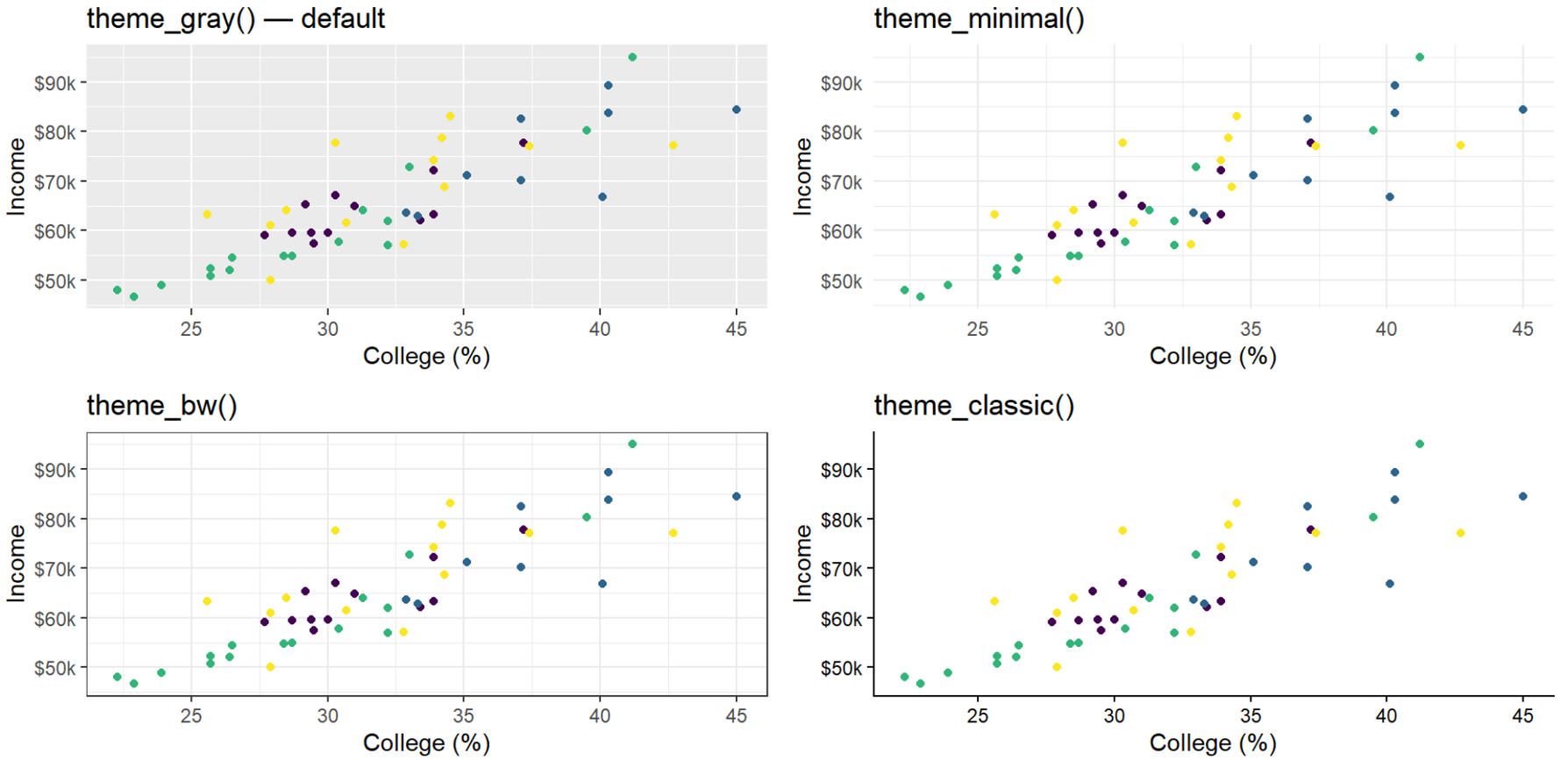
Step 3 — Color Palette

```
ggplot(state_data, aes(x = pct_college, y = median_income, color = region)) +
  geom_point(size = 2.5) +
  geom_smooth(aes(group = 1), method = "lm", se = FALSE,
              color = "gray40", linewidth = 0.8) +
  scale_color_viridis_d() +
  labs(
    title   = "Education Attainment and Household Income by State, 2022",
    x       = "Bachelor's Degree or Higher (%)",
    y       = "Median Household Income",
    color   = "Census Region",
    caption = "Source: ACS 5-Year Estimates 2022"
  )
```



Step 4 — Choose a Theme

Which theme fits your context?



Safe default: `theme_minimal()` — clean, works for reports and presentations. Use the **same theme** across every figure in your project for consistency.

Step 5 — Apply Theme

```
ggplot(state_data, aes(x = pct_college, y = median_income, color = region)) +
  geom_point(size = 2.5) +
  geom_smooth(aes(group = 1), method = "lm", se = FALSE,
              color = "gray40", linewidth = 0.8) +
  scale_color_viridis_d() +
  labs(
    title   = "Education Attainment and Household Income by State, 2022",
    x       = "Bachelor's Degree or Higher (%)",
    y       = "Median Household Income",
    color   = "Census Region",
    caption = "Source: ACS 5-Year Estimates 2022"
  ) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "right", panel.grid.minor = element_blank())
```

Step 5 — Fix the Axis Scale

ggplot2 pads axes by default — dead whitespace at the edges hides where the data actually lives. Zoom in with `coord_cartesian()`:

```
ggplot(state_data, aes(x = pct_college, y = median_income, color = region)) +
  geom_point(size = 2.5) +
  geom_smooth(aes(group = 1), method = "lm", se = FALSE,
              color = "gray40", linewidth = 0.8) +
  scale_color_viridis_d() +
  scale_y_continuous(labels = label_dollar(scale = 1e-3, suffix = "k")) +
  coord_cartesian(xlim = c(20, NA), ylim = c(40000, 100000)) +
  labs(
    title   = "Education Attainment and Household Income by State, 2022",
    x       = "Bachelor's Degree or Higher (%)",
    y       = "Median Household Income",
    color   = "Census Region",
    caption = "Source: ACS 5-Year Estimates 2022"
  ) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "right", panel.grid.minor = element_blank())
```

`coord_cartesian(xlim, ylim)` clips the view **without dropping data** — the trend line still uses all observations. Compare to `xlim()` / `ylim()`, which silently remove points outside the range.

Step 6 — In-Graph Text & Annotations

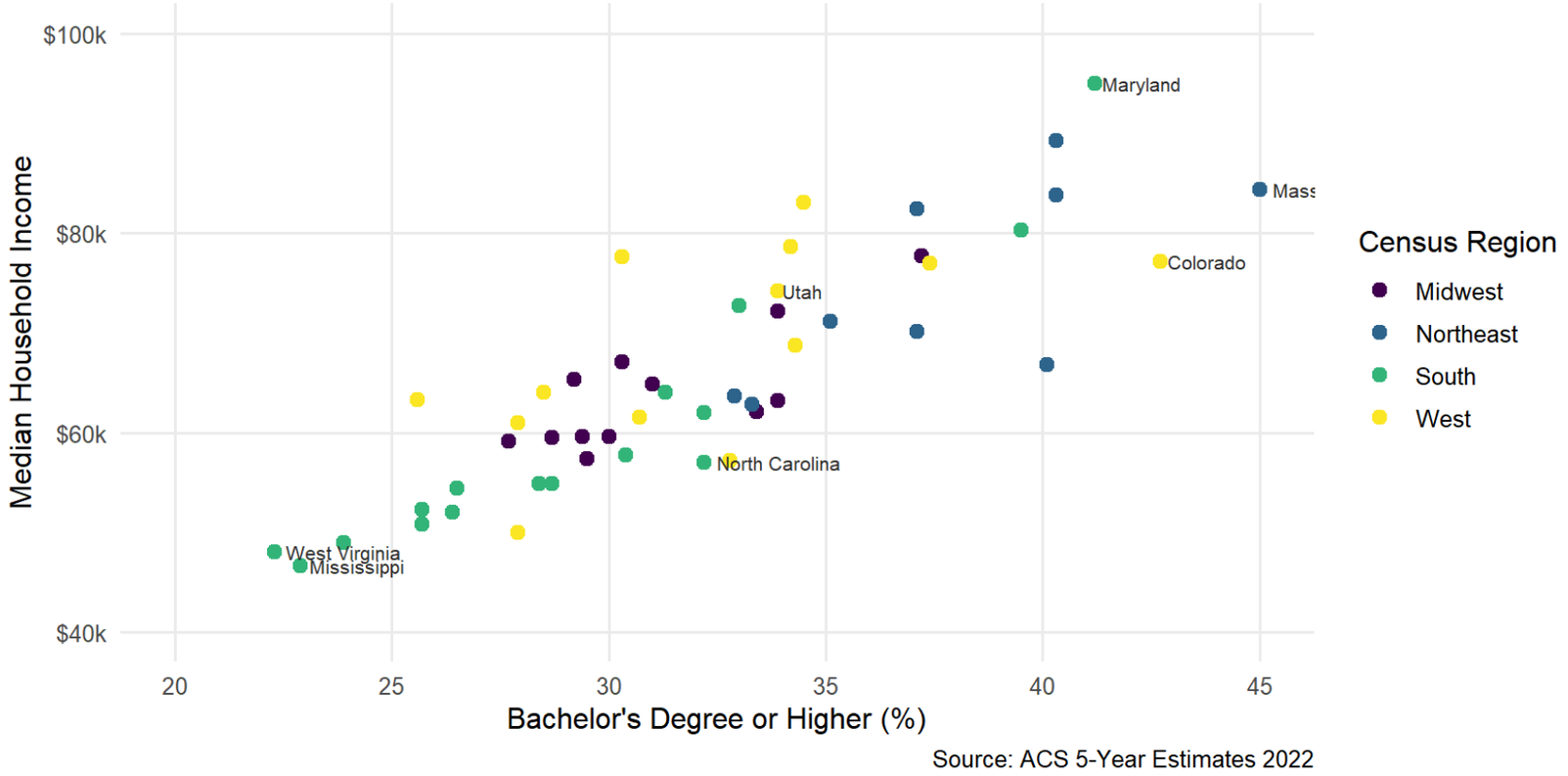
Four tools — use the right one for the job:

Function	Best for
`geom_text()`	Quick plain-text labels at data coordinates
`geom_text_repel()`	Plain text that auto-repositions to avoid overlap
`geom_label_repel()`	Text + background box, auto-repositioned
`annotate()`	Fixed callouts at exact coordinates (not tied to a data row)

```
# Base chart used for all four examples (defined once in setup)
base_ann <- ggplot(state_data, aes(x = pct_college, y = median_income, color = region)) +
  geom_point(size = 2.5) + scale_color_viridis_d() +
  scale_y_continuous(labels = label_dollar(scale = 1e-3, suffix = "k")) +
  coord_cartesian(xlim = c(20, NA), ylim = c(40000, 100000)) +
  labs(x = "Bachelor's (%)", y = "Median Income", color = "Region") +
  theme_minimal(base_size = 12)
```

`geom_text()` — Basic Labels

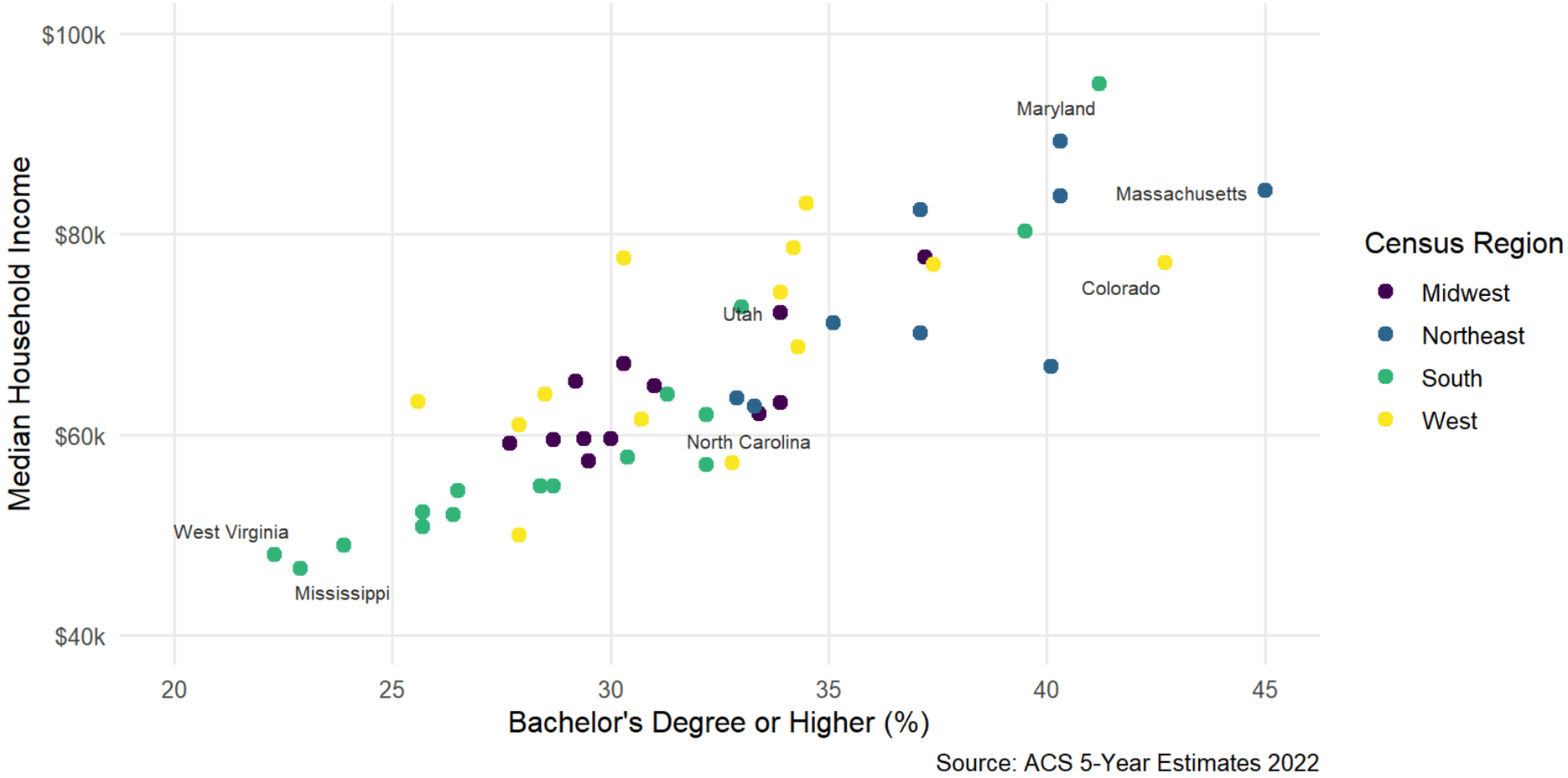
```
base_ann +
  geom_text(data = state_outliers, aes(label = NAME),
            size = 2.8, hjust = -0.1, color = "#2D2D2D")
```



North Carolina (~40% college, income < $70k) sits below the national trend — high education, lower wages.

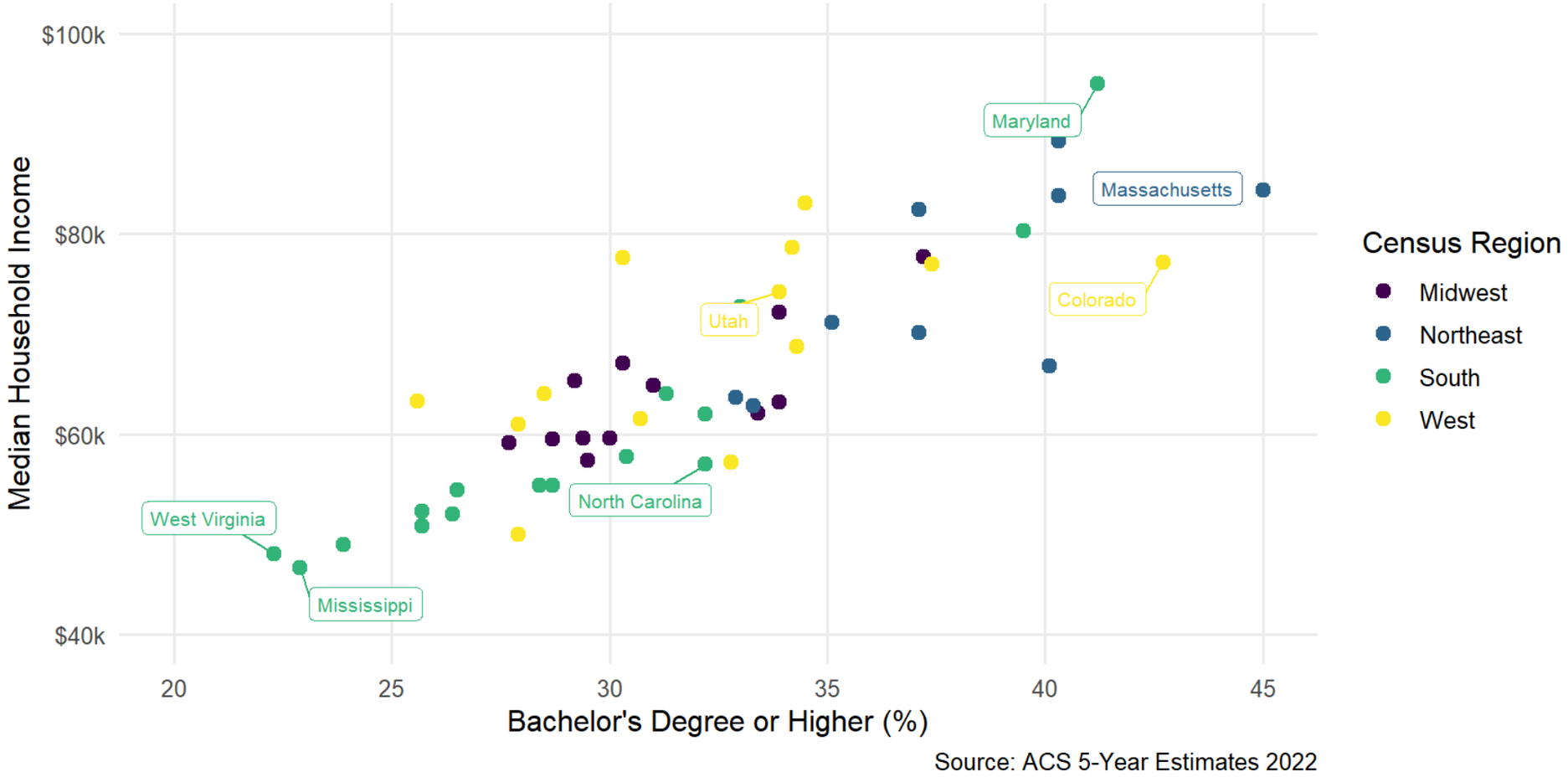
`geom_text_repel()` — Auto-repositioned Plain Text

```
base_ann +
  geom_text_repel(data = state_outliers, aes(label = NAME),
                  size = 2.8, box.padding = 0.4, color = "#2D2D2D")
```



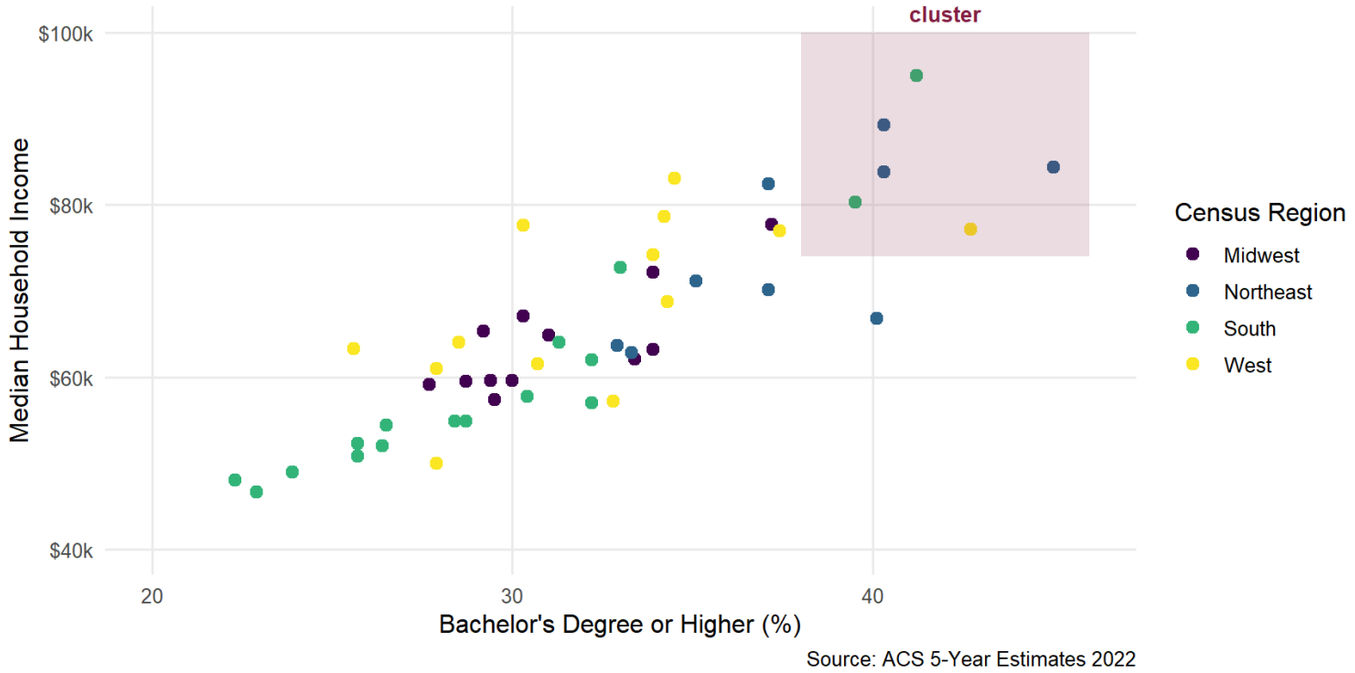
`geom_label_repel()` — Labels with Background Box

```
base_ann +
  geom_label_repel(data = state_outliers, aes(label = NAME),
                   size = 2.8, box.padding = 0.5, show.legend = FALSE)
```



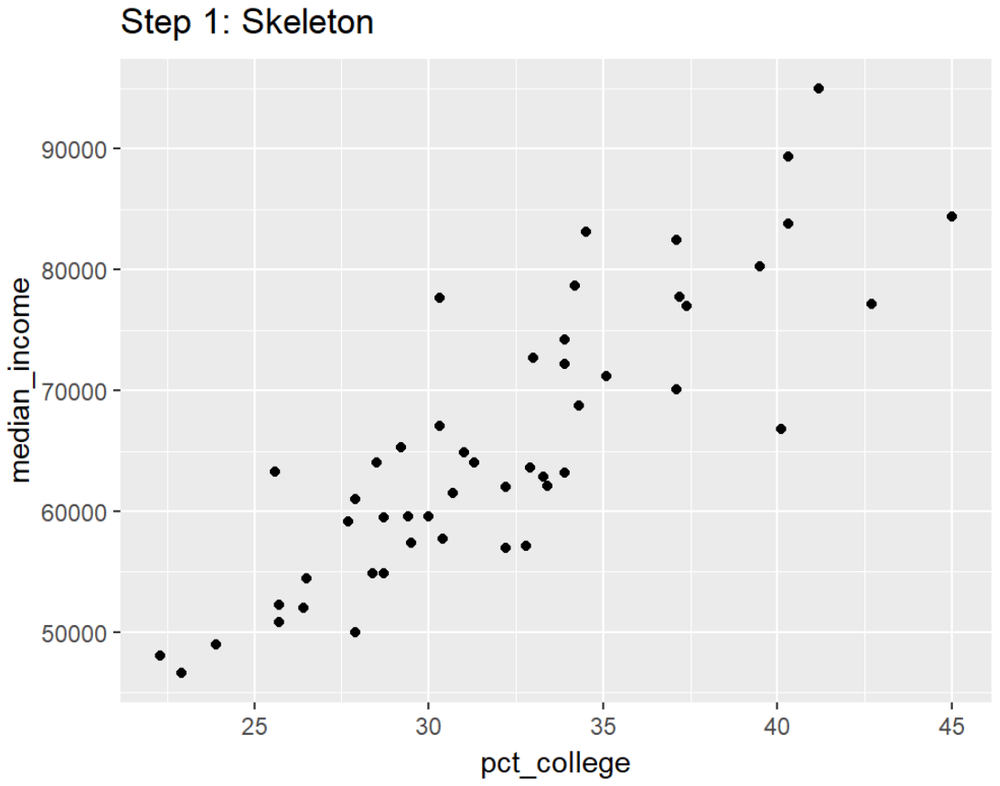
`annotate()` — Fixed Editorial Callouts

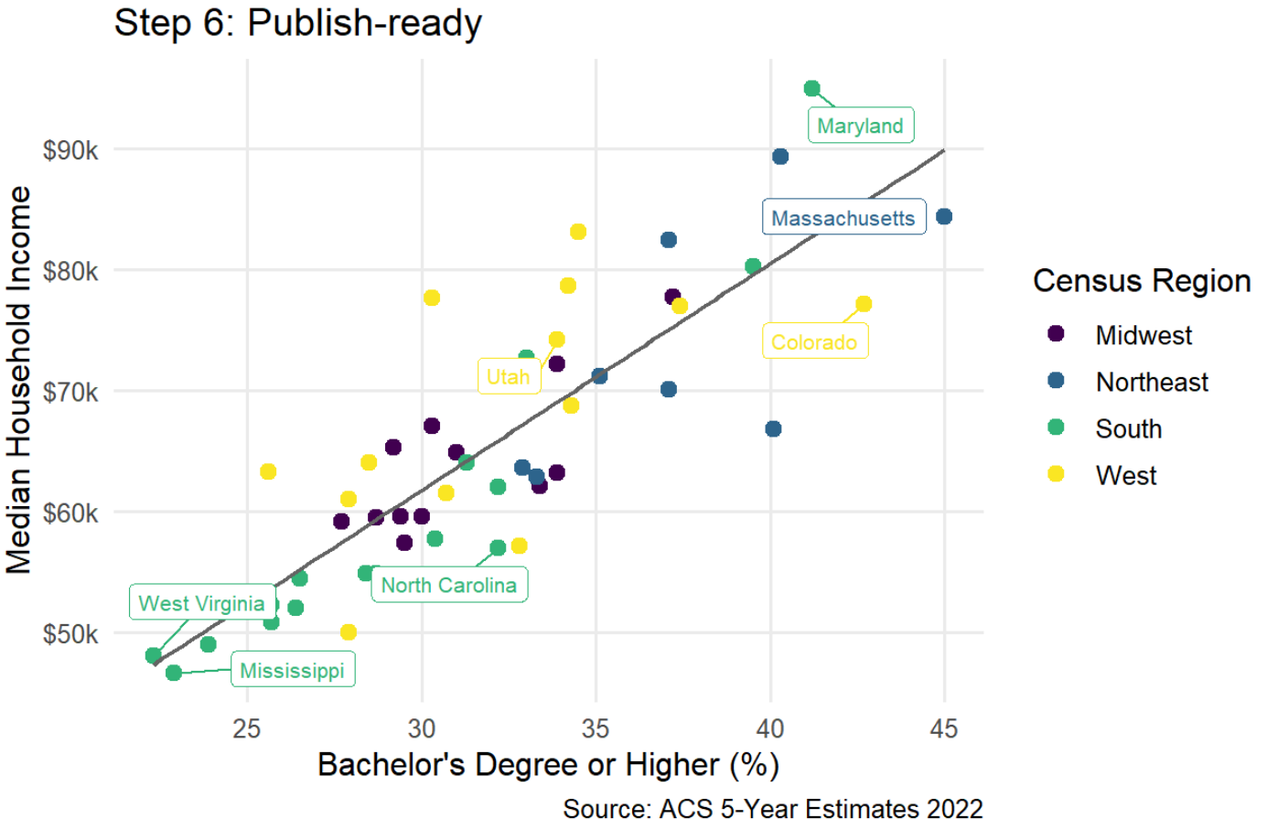
```
base_ann +
  annotate("rect", xmin = 38, xmax = 46, ymin = 74000, ymax = 100000,
           alpha = 0.15, fill = "#861F41") +
  annotate("text", x = 42, y = 104000,
           label = "High-ed, high-income\ncluster",
           size = 3.5, color = "#861F41", fontface = "bold")
```



`annotate()` is not data-driven — places shapes/text at **exact coordinates**. Use for region highlights, reference lines, or editorial notes.

Skeleton → Publish-Ready





→ Open `R/demos/dataviz01/01-workflow.R` to run all 7 steps interactively — run each section with Ctrl+Enter and watch the chart evolve.

Step 7 — Save with `ggsave()`

```r
final_plot <- ggplot(state_data, ...) + ...   # assign your complete plot

ggsave(
  filename = "figures/state_income_education_2022.png",
  plot     = final_plot,
  width    = 10,
  height   = 5.5,
  dpi      = 300
)
```

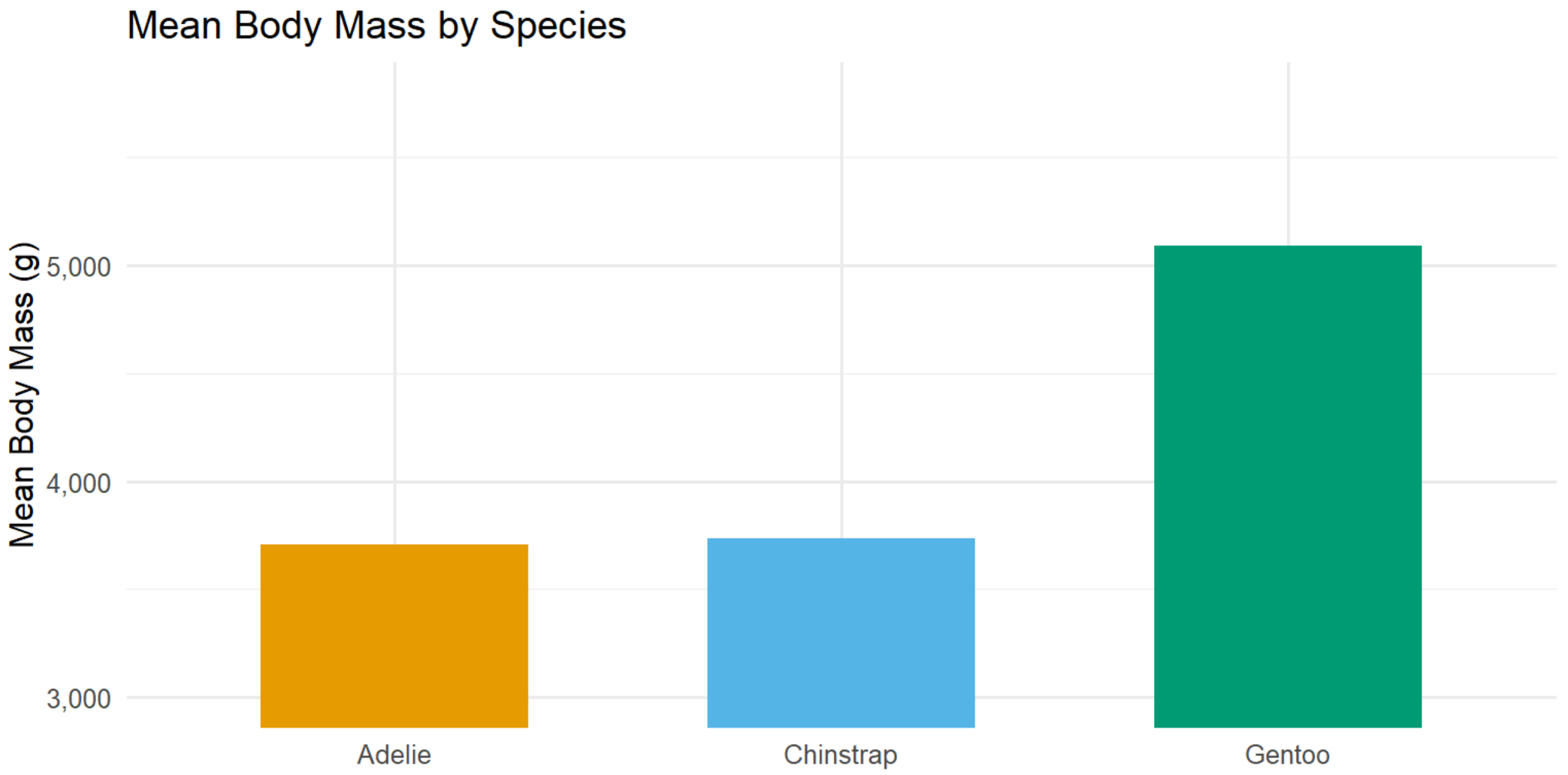
Format	Use when
PNG 300 dpi	Reports, Word, presentations
PDF	Journal submissions, LaTeX — vector, no blur

Always specify `width` and `height` explicitly — this controls the aspect ratio. After saving, zoom into the PNG at 100% to check it is not blurry (dpi = 300 is usually enough for print).

Spot the Issue

Spot the Issue — Plot 1

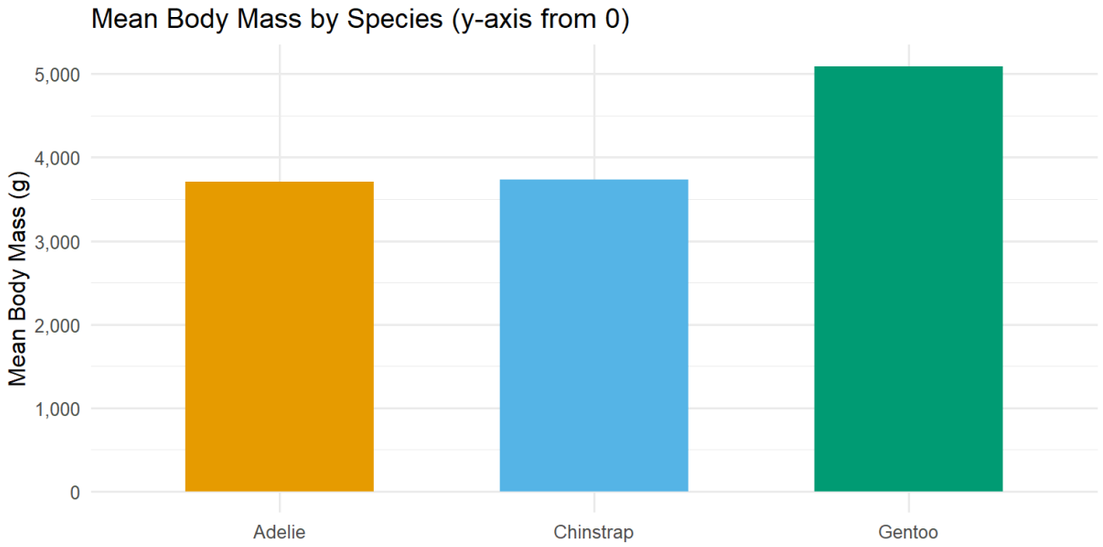
Scenario: Mean body mass (g) for three penguin species. *(Anything wrong?)*



Plot 1 — Answer

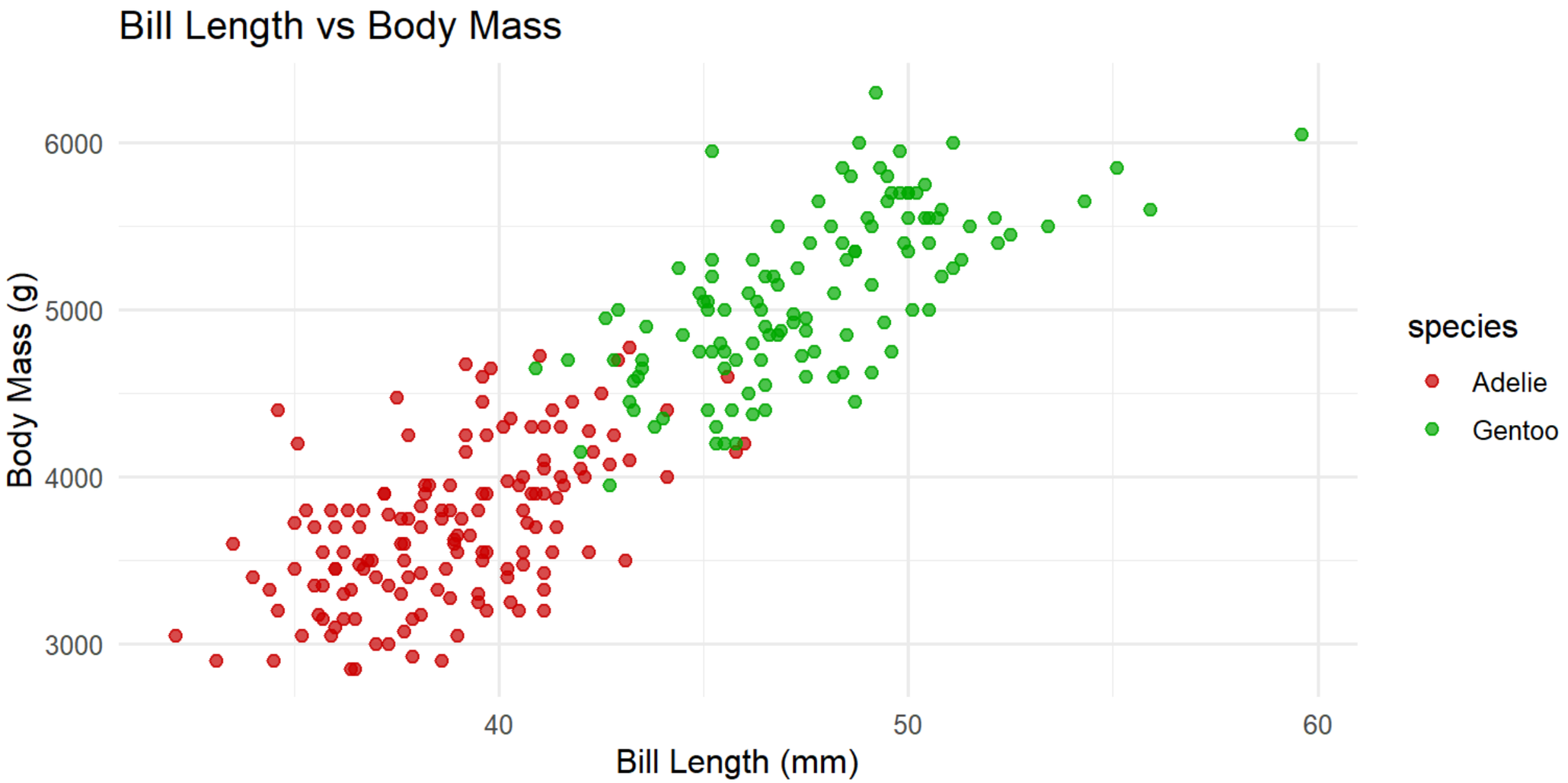
Problem: Y-axis starts at 3,000 g, not 0. Bars encode *length from zero* — truncating makes Gentoo look 3 times heavier than Adélie; the true ratio is ~1.4×.

```
penguins_c |> group_by(species) |>
  summarise(m = mean(body_mass_g)) |>
  ggplot(aes(x = species, y = m, fill = species)) +
  geom_col(width = 0.6) +
  scale_fill_manual(values = okabe3) +
  scale_y_continuous(labels = label_comma()) +   # no coord_cartesian cutoff
  labs(title = "Mean Body Mass by Species (y-axis from 0)",
       x = NULL, y = "Mean Body Mass (g)") +
  flaw_theme
```



Spot the Issue — Plot 2

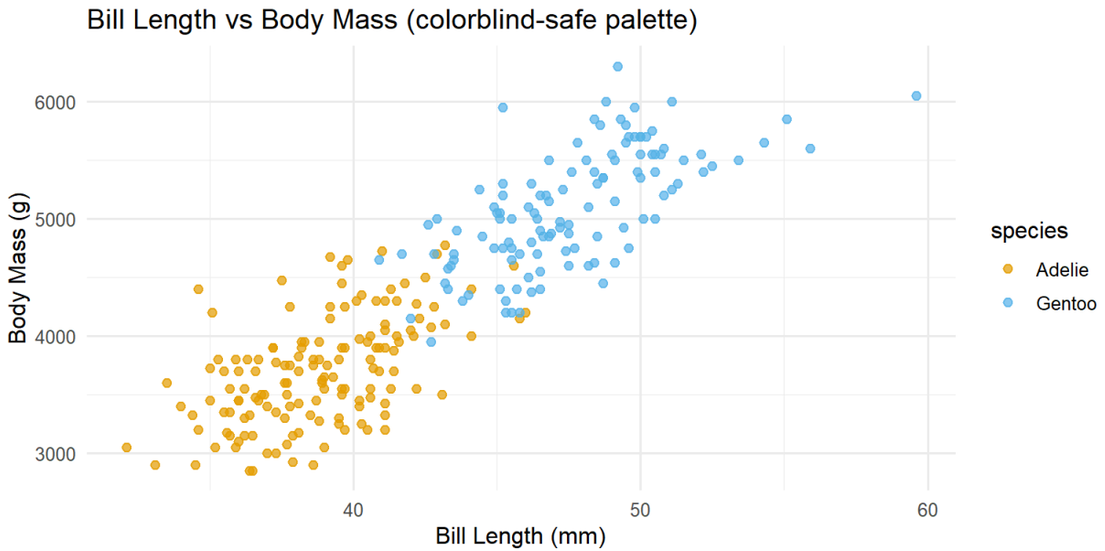
Scenario: Comparing bill length vs body mass for Adélie and Gentoo penguins. *(Anything wrong?)*



Plot 2 — Answer

Problem: Red + green fail for ~8% of men (red-green color vision deficiency). The two groups become indistinguishable.

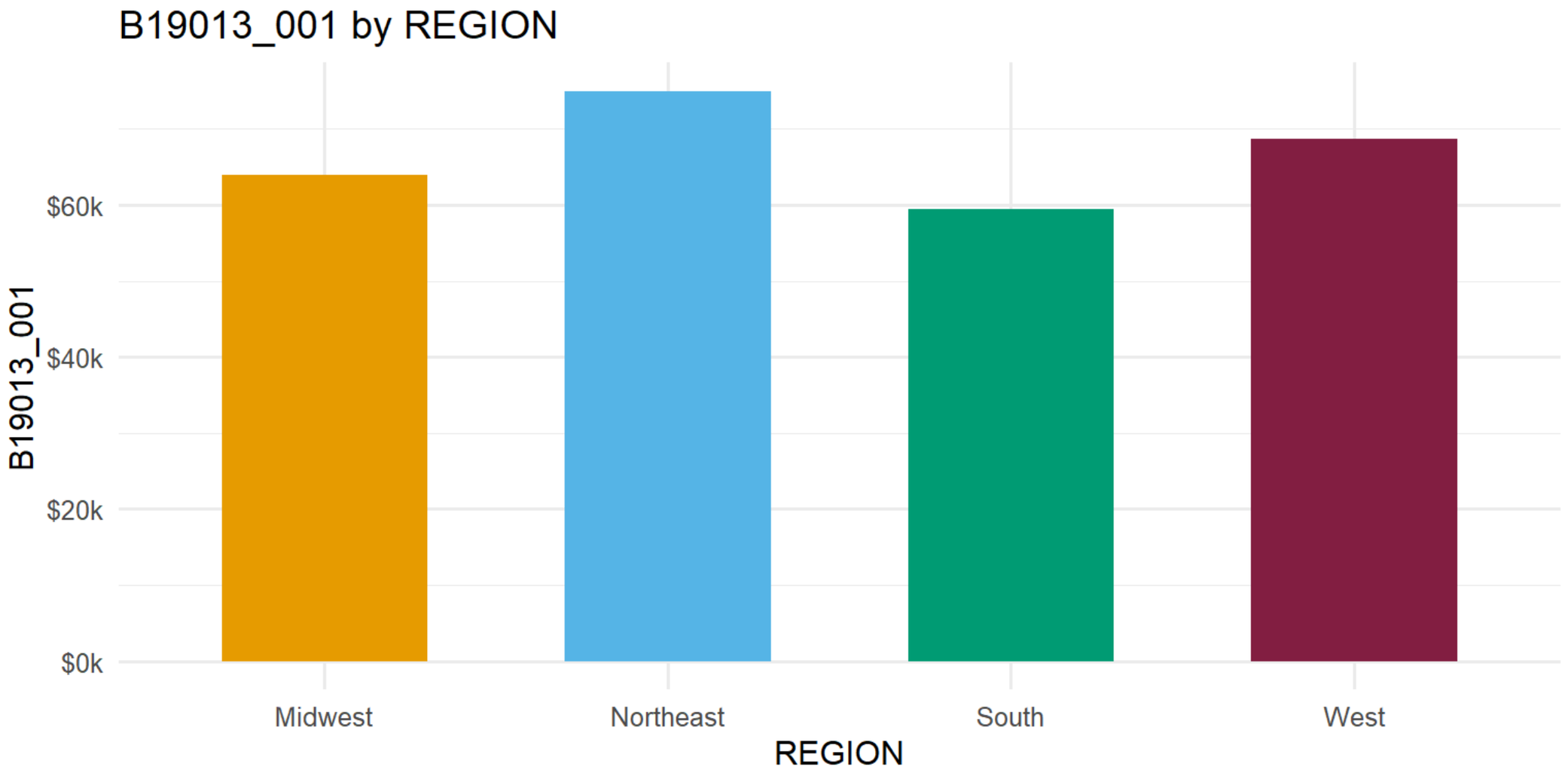
```
penguins_c |> filter(species != "Chinstrap") |>
  ggplot(aes(x = bill_length_mm, y = body_mass_g, color = species)) +
  geom_point(size = 2, alpha = 0.7) +
  scale_color_manual(values = c("Adelie" = "#E69F00", "Gentoo" = "#56B4E9")) +
  labs(title = "Bill Length vs Body Mass (colorblind-safe palette)",
       x = "Bill Length (mm)", y = "Body Mass (g)") +
  flaw_theme + theme(legend.position = "right")
```



Okabe-Ito (`#E69F00`, `#56B4E9`, `#009E73`) and `scale_color_viridis_d()` are safe for all common color vision types.

Spot the Issue — Plot 3

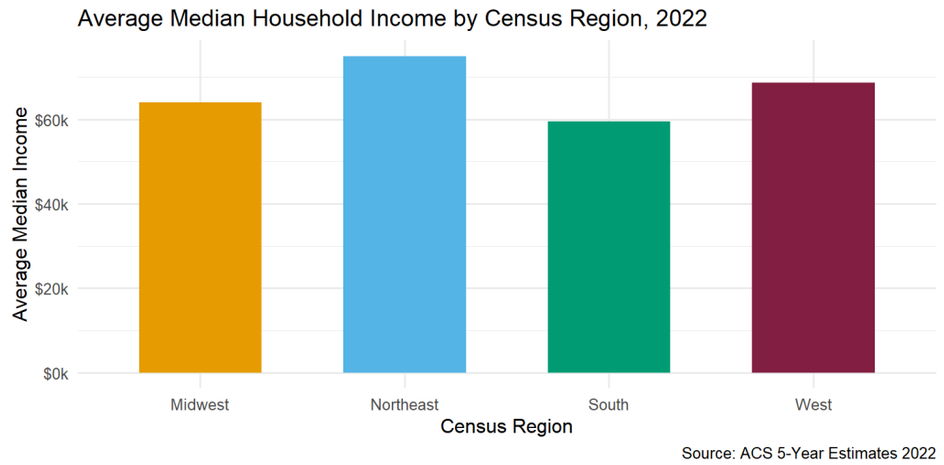
Scenario: Average income by Census region, pulled straight from the ACS API. *(Anything wrong?)*



Plot 3 — Answer

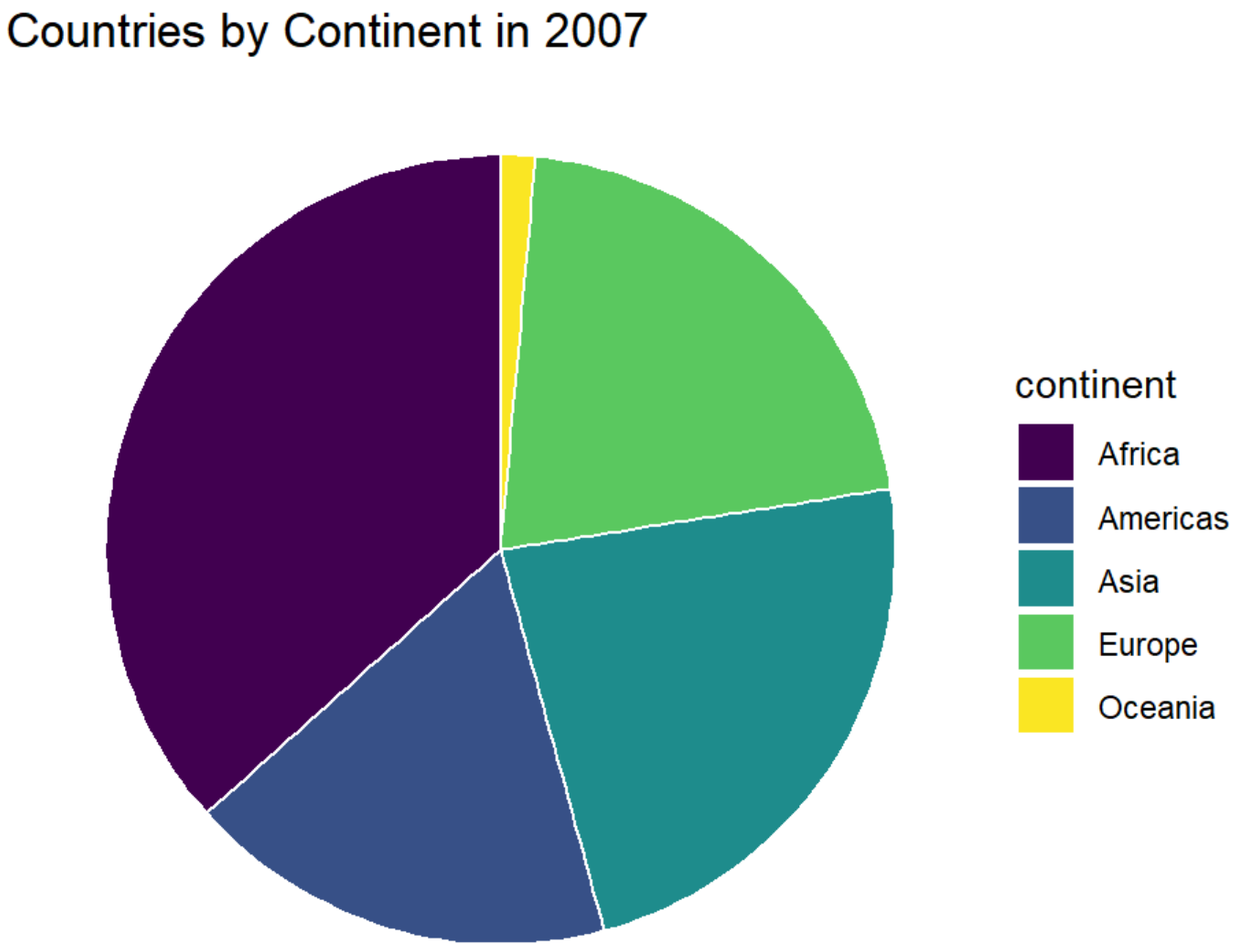
Problem: Raw API variable codes as axis labels. `B19013_001` means nothing to a reader. Always rename with `labs()` or rename columns before plotting.

```
state_data |>
  group_by(region) |>
  summarise(avg_income = mean(median_income)) |>
  ggplot(aes(x = region, y = avg_income, fill = region)) +
  geom_col(width = 0.6) +
  scale_fill_manual(values = c("#E69F00", "#56B4E9", "#009E73", "#861F41")) +
  scale_y_continuous(labels = label_dollar(scale = 1e-3, suffix = "k")) +
  labs(title = "Average Median Household Income by Census Region, 2022",
       x = "Census Region", y = "Average Median Income",
       caption = "Source: ACS 5-Year Estimates 2022") +
  flaw_theme
```



Spot the Issue — Plot 4

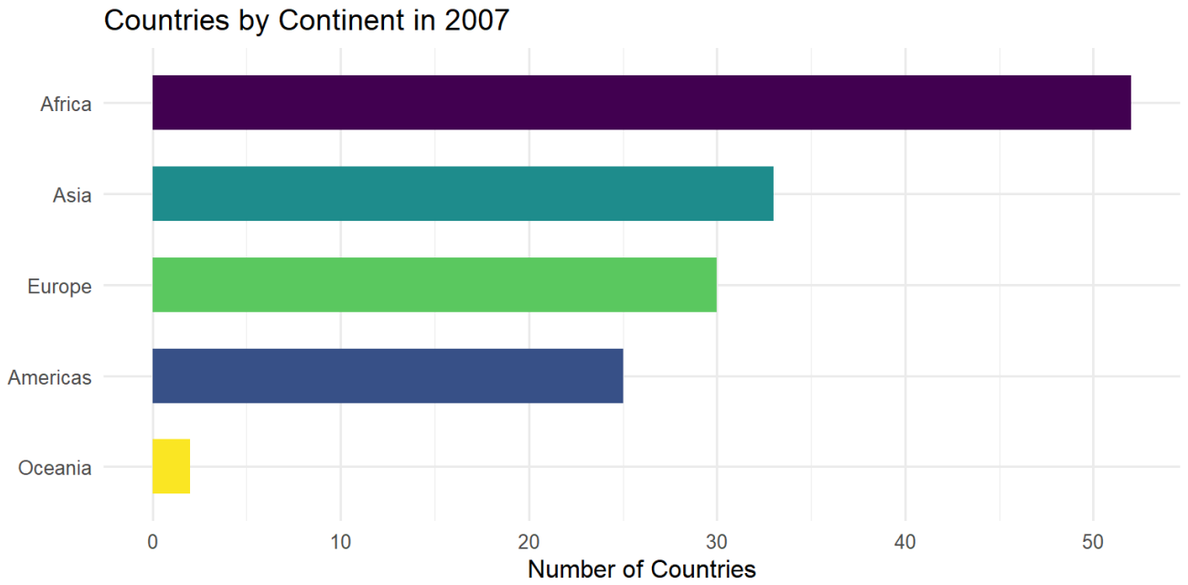
Scenario: Number of countries per continent in the 2007 gapminder dataset. *(Anything wrong?)*



Plot 4 — Answer

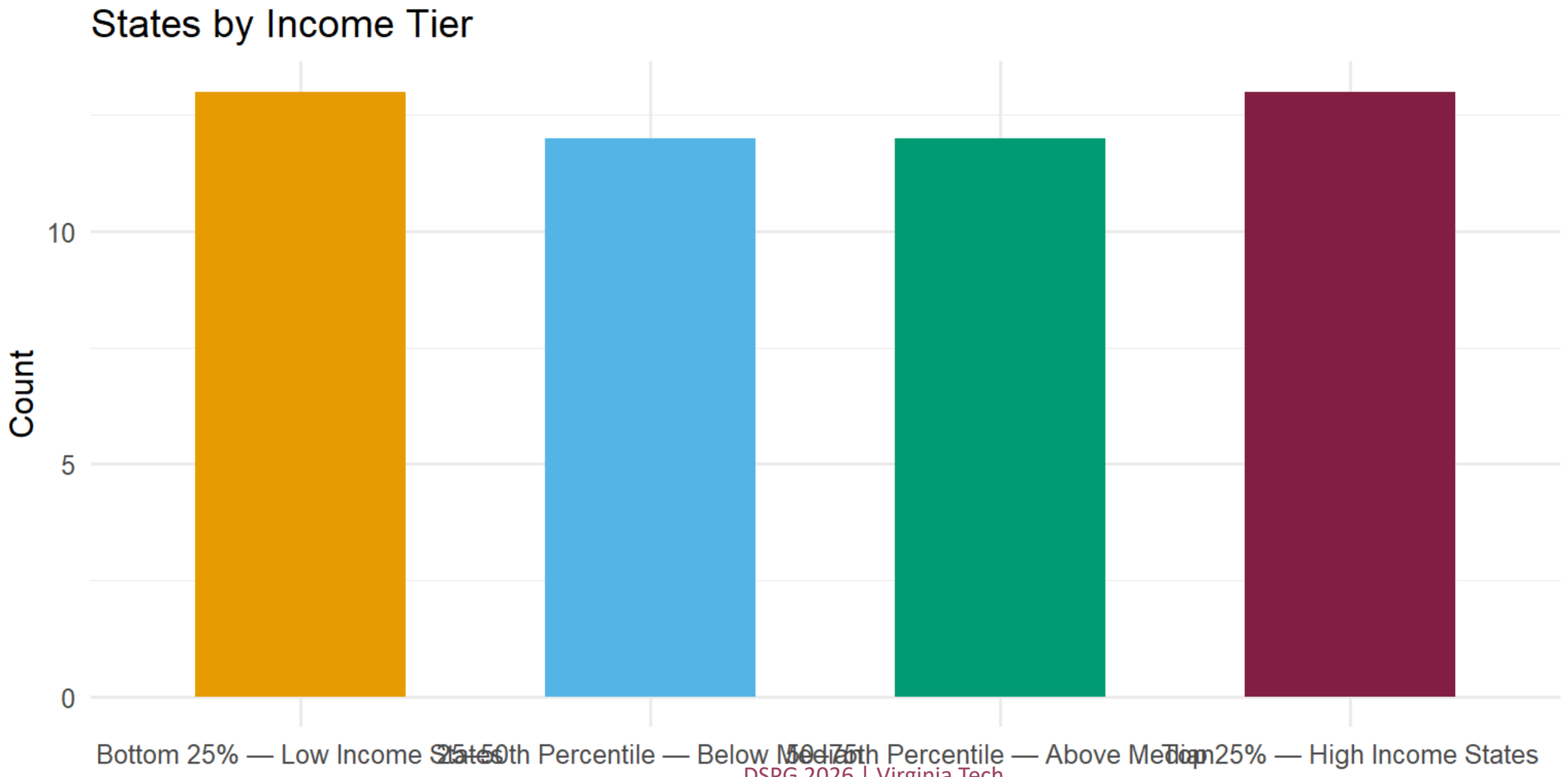
Problem: Pie charts require judging angles — humans are poor at this, especially when slices are similar. A sorted bar is always more readable.

```
gap07 |> count(continent) |>
  ggplot(aes(x = reorder(continent, n), y = n, fill = continent)) +
  geom_col(width = 0.6) +
  scale_fill_viridis_d() +
  coord_flip() +
  labs(title = "Countries by Continent in 2007",
       x = NULL, y = "Number of Countries") +
  flaw_theme
```



Spot the Issue — Plot 5

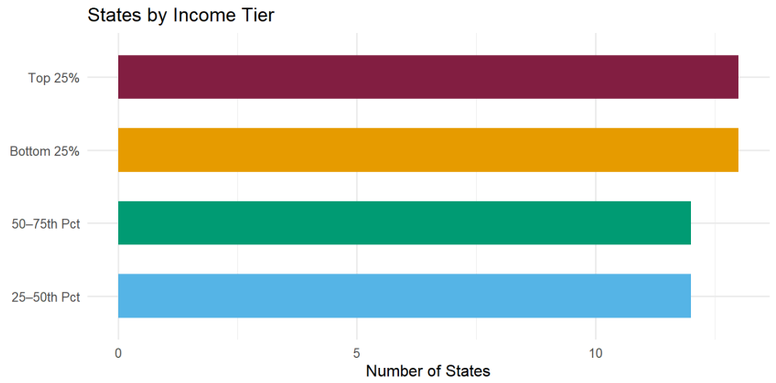
Scenario: Distribution of species count by region — student copied ggplot defaults and did not touch `theme()`. *(Anything wrong?)*



Plot 5 — Answer

Problem: Long x-axis labels overlap and become unreadable. Fix with `coord_flip()` or `theme(axis.text.x = element_text(angle = 30, hjust = 1))`.

```
state_data |>
  mutate(income_tier = cut(median_income,
    breaks = quantile(median_income, c(0, .25, .5, .75, 1)),
    labels = c("Bottom 25%", "25–50th Pct", "50–75th Pct", "Top 25%"),
    include.lowest = TRUE)) |>
  count(income_tier) |>
  ggplot(aes(x = reorder(income_tier, n), y = n, fill = income_tier)) +
  geom_col(width = 0.6) +
  scale_fill_manual(values = c("#E69F00", "#56B4E9", "#009E73", "#861F41")) +
  coord_flip() +
  labs(title = "States by Income Tier",
       x = NULL, y = "Number of States") +
  flaw_theme
```



Static Maps with `geom_sf`

Why `geom_sf`?

You already know `leaflet` for interactive maps. `geom_sf` is different:

	`geom_sf` (ggplot2)	`leaflet`
Output	Static — PNG, PDF	Interactive HTML
Grammar	Same as all other ggplot2	Separate syntax
Use for	Reports, papers, posters	Dashboards, web apps
Theming	`theme()`, `labs()`, `ggsave()`	Own legend/layout system

> `geom_sf` is just another geom. Everything you learned today — color palettes, `labs()`, `theme_void()`, `ggsave()` — applies directly to maps.

State SF Data via `tidycensus`

Add `geometry = TRUE` to any `get_acs()` call — you get data **and** map-ready geometries in one step:

```
library(tidycensus)

state_sf <- get_acs(
  geography = "state",
  variables = c(median_income = "B19013_001"),
  year      = 2022,
  geometry  = TRUE          # ← same get_acs() you already know, + geometry
)
```

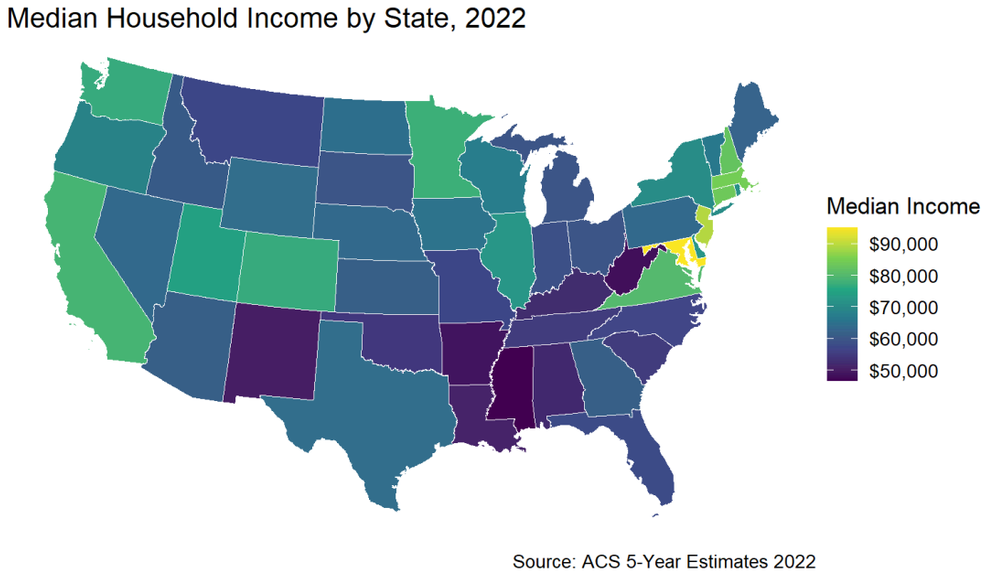
```
state_sf |>
  select(NAME, median_income, geometry) |>
  head(4)
```

```
Simple feature collection with 4 features and 2 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -124.4096 ymin: 31.3323 xmax: -81.96497 ymax: 45.94545
Geodetic CRS:  NAD83
          NAME median_income                       geometry
1   New Mexico         50003 MULTIPOLYGON (((-109.0502 3...
2 South Dakota         59533 MULTIPOLYGON (((-104.0579 4...
3   California         78672 MULTIPOLYGON (((-118.6044 3...
4     Kentucky         52295 MULTIPOLYGON (((-89.40565 3...
```

- The `geometry` column holds the state boundary shapes — no shapefile download needed
- Use any `dplyr` verb on it exactly as you would a regular tibble
- `geom_sf()` reads the geometry column automatically — no `x`/`y` mapping needed

State Income Choropleth

```
ggplot(state_sf) +
  geom_sf(aes(fill = median_income), color = "white", linewidth = 0.2) +
  scale_fill_viridis_c(
    name   = "Median Income",
    labels = label_dollar()
  ) +
  coord_sf(crs = 5070) +
  theme_void(base_size = 13) +
  labs(
    title   = "Median Household Income by State, 2022",
    caption = "Source: ACS 5-Year Estimates 2022"
  )
```



`tidycensus` — One Call Gets Data + Map

```
library(tidycensus)

va_income <- get_acs(
  geography = "county",
  state     = "VA",
  variables = "B19013_001",   # median household income
  year      = 2022,
  geometry  = TRUE            # returns an sf object directly — no separate download
)

ggplot(va_income) +
  geom_sf(aes(fill = estimate), color = "white", linewidth = 0.2) +
  scale_fill_viridis_c(
    name   = "Median Income (USD)",
    labels = label_dollar()
  ) +
  coord_sf(crs = 5070) +     # Albers equal-area — better for US maps
  theme_void(base_size = 13) +
  labs(
    title   = "Median Household Income by County, Virginia 2022",
    caption = "Source: American Community Survey 2022"
  )
```

`geometry = TRUE` returns an `sf` object directly — no shapefile, no join, one step.

Virginia County Income — Output

Joining Your Own Data to a Map

```
# Pattern: join any project data frame to county geometries
library(tigris)

counties_sf <- counties(state = "VA", cb = TRUE, year = 2022)

my_data <- read_csv("data/my_county_metrics.csv")
# my_data must have a GEOID column (5-digit FIPS code)

counties_joined <- counties_sf |>
  left_join(my_data, by = "GEOID")

ggplot(counties_joined) +
  geom_sf(aes(fill = your_metric), color = "white", linewidth = 0.2) +
  scale_fill_viridis_c() +
  coord_sf(crs = 5070) +
  theme_void()
```

Same `left_join()` you already know — the only difference is the data frame has a geometry column.

Saving a Map

```
my_map <- ggplot(va_income) +
  geom_sf(aes(fill = estimate)) +
  scale_fill_viridis_c(labels = label_dollar()) +
  coord_sf(crs = 5070) +
  theme_void() +
  labs(title = "Median Income, Virginia 2022")

ggsave(
  "figures/va_income_map.png",
  plot   = my_map,
  width  = 10,
  height = 6,
  dpi    = 300
)
```

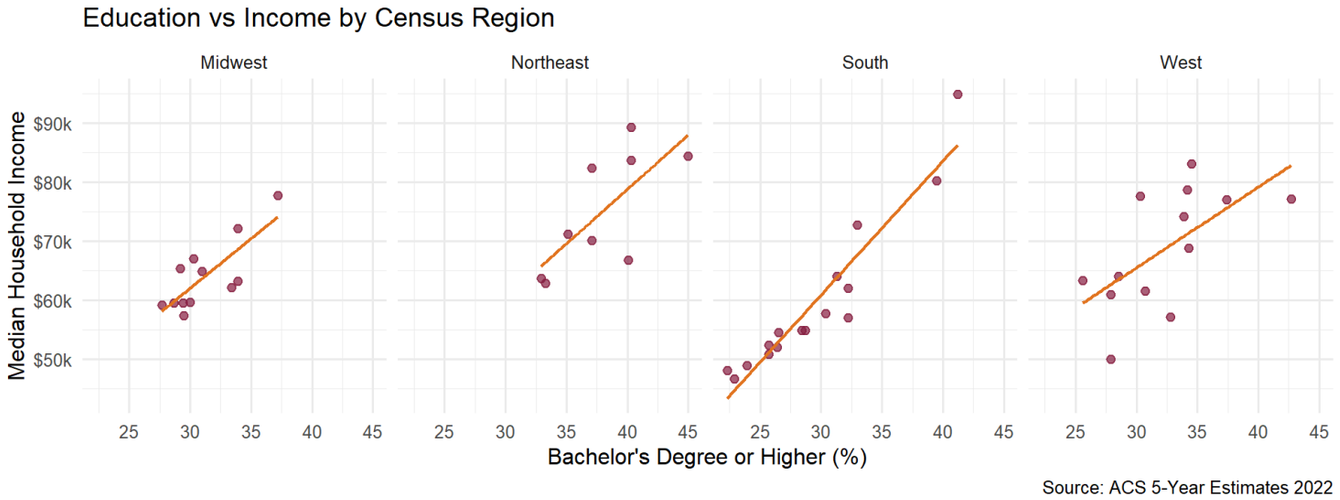
Exactly the same as any other ggplot — `ggsave()` with explicit width, height, dpi.

Faceting & Multi-panel Layouts

`facet_wrap()` — Small Multiples

Q: Does the income–education relationship differ across Census regions?

```
ggplot(state_data, aes(x = pct_college, y = median_income)) +
  geom_point(color = "#861F41", size = 2, alpha = 0.7) +
  geom_smooth(method = "lm", se = FALSE, color = "#E5751F", linewidth = 0.8) +
  scale_y_continuous(labels = label_dollar(scale = 1e-3, suffix = "k")) +
  facet_wrap(~ region, nrow = 1) +
  labs(title = "Education vs Income by Census Region",
       x = "Bachelor's Degree or Higher (%)", y = "Median Household Income",
       caption = "Source: ACS 5-Year Estimates 2022") +
  theme_minimal(base_size = 12)
```



Use `scales = "free_x"` or `"free"` to let axis ranges differ across panels.

`facet_grid()` — Two-way Panels

Q: Is the gender gap in college attainment consistent across regions?

```
# Requires tidycensus — run once and save locally
library(tidycensus)
vars_sex <- c(inc_median  = "B19013_001",
              ed_total    = "B15002_001",
              col_male    = "B15002_015",
              col_female  = "B15002_032")

state_sex <- get_acs(geography = "state", variables = vars_sex,
                     year = 2022, output = "wide") |>
  mutate(pct_male   = col_maleE   / (ed_totalE / 2) * 100,
         pct_female = col_femaleE / (ed_totalE / 2) * 100) |>
  left_join(state_data |> select(NAME, region), by = "NAME") |>
  filter(!is.na(region)) |>
  pivot_longer(cols = c(pct_male, pct_female),
               names_to = "sex", values_to = "pct_college",
               names_prefix = "pct_") |>
  mutate(sex = str_to_title(sex))

ggplot(state_sex, aes(x = pct_college, y = inc_medianE)) +
  geom_point(color = "#861F41", size = 1.5, alpha = 0.6) +
  scale_y_continuous(labels = label_dollar(scale = 1e-3, suffix = "k")) +
  facet_grid(sex ~ region) +
```

Sex × Region — Output

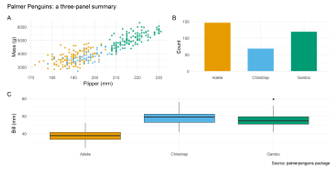
`patchwork` — Combining Different Plots

```
p_scatter <- ggplot(penguins |> drop_na(),
                    aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point(alpha = 0.7) +
  scale_color_manual(values = c("#E69F00", "#56B4E9", "#009E73")) +
  theme_minimal(base_size = 11) + theme(legend.position = "none") +
  labs(x = "Flipper (mm)", y = "Mass (g)")

p_bar <- ggplot(penguins |> drop_na() |> count(species),
                aes(x = species, y = n, fill = species)) +
  geom_col(width = 0.6) +
  scale_fill_manual(values = c("#E69F00", "#56B4E9", "#009E73")) +
  theme_minimal(base_size = 11) + theme(legend.position = "none") +
  labs(x = NULL, y = "Count")

p_box <- ggplot(penguins |> drop_na(),
                aes(x = species, y = bill_length_mm, fill = species)) +
  geom_boxplot() +
  scale_fill_manual(values = c("#E69F00", "#56B4E9", "#009E73")) +
  theme_minimal(base_size = 11) + theme(legend.position = "none") +
  labs(x = NULL, y = "Bill (mm)")

((p_scatter | p_bar) / p_box) +
```



`patchwork` Operators

Operator	What it does
`p1 | p2`	Side by side
`p1 / p2`	Stacked vertically
`(p1 | p2) / p3`	Two on top, one full-width below
`plot_layout(widths = c(2, 1))`	Left panel twice as wide as right
`plot_layout(guides = "collect")`	Move all legends to one shared legend
`plot_annotation(tag_levels = "A")`	Add panel labels A, B, C …

```
# Unequal widths: scatter gets 2/3 of the space, bar + box share 1/3
(p_scatter | (p_bar / p_box)) +
  plot_layout(widths = c(2, 1)) +
  plot_annotation(
    title      = "Palmer Penguins: a three-panel summary",
    caption    = "Source: palmerpenguins package",
    tag_levels = "A"
  )
```

→ Open `R/demos/dataviz01/02-patchwork.R` to try all four patchwork layout operators (`|`, `/`, compound, and unequal widths) interactively.

Q: Do states with higher poverty rates have lower median incomes?

```r
library(tidycensus)

state_pov <- get_acs(
  geography = "state",
  variables = c(pov_count  = "B17001_002",   # people below poverty line
                pov_total  = "B17001_001",   # total people w/ poverty status
                inc_median = "B19013_001"),  # median household income
  year = 2022, output = "wide"
) |>
  mutate(pov_rate = pov_countE / pov_totalE) |>
  left_join(state_data |> select(NAME, region), by = "NAME")

# Tasks:
# 1. Scatter: pov_rate (x) vs inc_medianE (y), color = region
# 2. scale_x_continuous(labels = label_percent())
# 3. scale_y_continuous(labels = label_dollar(scale = 1e-3, suffix = "k"))
# 4. Add geom_smooth(method = "lm", se = FALSE, color = "gray40")
# 5. Label 3 highest-poverty + 3 lowest-poverty states with geom_label_repel()
# 6. Apply your full 7-step workflow (color, theme, axis, title, ggsave)
# 7. One sentence: what does this scatter tell you?
```

Your Turn — Answer

```
state_pov |>
  ggplot(aes(x = pov_rate, y = inc_medianE, color = region)) +
  geom_point(size = 2.5, alpha = 0.8) +
  geom_smooth(aes(group = 1), method = "lm", se = FALSE,
              color = "gray40", linewidth = 0.9) +
  geom_label_repel(
    data = state_pov |>
      slice_max(pov_rate, n = 3) |>
      bind_rows(state_pov |> slice_min(pov_rate, n = 3)),
    aes(label = NAME), size = 2.8, box.padding = 0.4, show.legend = FALSE
  ) +
  scale_x_continuous(labels = label_percent()) +
  scale_y_continuous(labels = label_dollar(scale = 1e-3, suffix = "k")) +
  scale_color_viridis_d() +
  coord_cartesian(ylim = c(40000, 100000)) +
  labs(
    title   = "Poverty Rate vs Median Household Income by State, 2022",
    x       = "Poverty Rate",
    y       = "Median Household Income",
    color   = "Census Region",
    caption = "Source: ACS 5-Year Estimates 2022"
  ) +
```

Yes — states with higher poverty rates have significantly lower median incomes. The scatterplot shows a strong negative relationship (r ≈ −0.80): high-poverty states cluster in the South (Mississippi, Louisiana, West Virginia), while low-poverty states cluster in the Northeast and West (Maryland, New Hampshire, Colorado). Poverty rate explains roughly 64% of the variance in state median income — a strong association, though not proof of causation.

Key Takeaways

Key Takeaways

- **Every plot needs one sentence** — if you cannot write it, the chart is not done
- **7-step workflow** — Skeleton → Labels → Color → Theme → Axis → In-graph text → Save
- **Color has rules** — Okabe-Ito / viridis_d for categories; viridis_c for sequential; RdBu for diverging; max 5–6 colors; always colorblind-safe
- **Match chart to data situation** — continuous distribution → histogram; categorical × continuous → boxplot/violin; two continuous → scatter; time + composition → stacked area; two categorical → heatmap
- **Faceting** splits one plot by a categorical variable; `patchwork` combines different plot types
- `ggsave()` with explicit width and height — set the aspect ratio, then zoom in to check the saved PNG is not blurry

Resources

Essential References

📘 R for Data Science — Data Visualization — Wickham, Çetinkaya-Rundel & Grolemund; the canonical ggplot2 guide

💻 r-dataviz-webinar (GitHub) — companion code and examples for this workshop

- ggplot2 documentation — reference for every geom, scale, and theme element
- patchwork — multi-panel layout
- ggrepel — non-overlapping labels
- scales — axis formatting helpers
- ColorBrewer — colorblind-safe palettes